

# Задача о минимальном k-разрезе

## Аннотация

В данной работе исследуется задача о минимальном k-разрезе (Min k-Cut) — нахождении в неориентированном взвешенном графе множества ребер минимального суммарного веса, удаление которых приводит к распадению графа не менее чем на k компонент связности.

В первой части работы доказывается NP-полнота задачи Min k-Cut путём построения цепочки полиномиальных сведений:

$$\mathbf{NAE-3SAT} \rightarrow \mathbf{Max\ Cut} \rightarrow \mathbf{Min\ Cut\ into\ Bounded\ Sets} \rightarrow \mathbf{Min\ k-Cut}$$

Во второй части рассматривается приближённый полиномиальный алгоритм EFFICIENT [SV], основанный на дереве Гомори-Ху и жадной стратегии выбора минимальных разрезов. Приводится оценки качества работы алгоритма и вычислительной сложности.

В третьей части приведены реализация алгоритма EFFICIENT и тестирование.

## Введение

Задача о минимальном  $k$ -разрезе (Min k-Cut) состоит в следующем:

Дан неориентированный граф с неотрицательными весами ребер и целое число  $k$ ; требуется найти множество ребер минимального суммарного веса, удаление которых разбивает граф не менее, чем на  $k$  компонент связности. Задача находит применение в VLSI-проектировании, кластеризации и сегментации изображений.

Min k-Cut является NP-полной задачей [GJ]. Для фиксированного  $k$  существуют полиномиальные точные алгоритмы [GH], однако их сложность экспоненциально зависит от  $k$ .

В 1991 году Саран и Вазирани [SV] предложили алгоритм EFFICIENT, находящий  $k$ -разрез, вес которого не более чем в  $(2 - \frac{2}{k})$  раз превосходит оптимальный. Алгоритм использует дерево Гомори-Ху [GoHu] — структуру, позволяющую за  $n - 1$  вызовов *max flow* получить набор из  $n - 1$  разреза, достаточный для построения приближённого решения. Оценка  $(2 - \frac{2}{k})$  является неулучшаемой для данного класса жадных алгоритмов.

## Принадлежность NPC

### Обозначения:

- Min k-Cut** :=  $\{(G = (V, E), w, C, k) \mid \exists \tilde{E} \subset E \text{ стоимости } \leq C, \text{ что граф } (V, E \setminus \tilde{E}) \text{ распадается на } \geq k \text{ компонент связности}\}$
- Min Cut into Bounded Sets** :=  $\{(G = (V, E), B, w, C) \mid \exists \text{ множества вершин } V_1, V_2 : V = V_1 \sqcup V_2; |V_1|, |V_2| \leq B \text{ и сумма весов ребер между } V_1 \text{ и } V_2 \leq C\}$
- Max Cut** :=  $\{(G = (V, E), C)\} \exists \text{ множество вершин } \tilde{V}, \text{ что в графе } G \text{ между } V \text{ и } V \setminus \tilde{V} \text{ хотя бы } C \text{ ребер}\}$
- NAE-3SAT** := {множество 3-KНФ, у которых существует набор, что в каждом дизъюнкте формулы встречается как 0, так и 1}

### Договоренность

Для формальности будем считать  $k \leq |V|$  в Min k-Cut

### Замечание:

**Min Cut into Bounded Sets** в данном проекте является вариантом задачи ND17 из [GJ] (без фиксации вершин  $s, t$ )

### Доказательство принадлежности NP

Предъявим сертификат  $y$  и верификатор  $V(x, y)$ :

- $y$  - набор ребер  $(\tilde{E}, \text{ список вершин для каждой компоненты})$  - набор из  $k + 1$  элемента
- $V(x, y) = \begin{cases} \text{Распаковывает } x, y, \text{ где } x = (G, w, C) \\ \text{Проверяет, что сумма весов в } \tilde{E} \leq C \\ \text{Для каждой пары компонент проверяет, что между их вершинами нет ребер} \end{cases}$

### Корректность доказательства

Корректность соответствия  $l \in L \Leftrightarrow \exists y \ V(x, y) = 1$  понятна по построению.

Оценка сложности:  $|G| = poly(|V|)$ .

Нетрудно убедиться, что каждый шаг верификатора также выполняется за  $poly(|V|)$ :

- За  $O(|G| + |E| + 1)$
- За  $O(|E|)$
- За  $O(k^2 \times |V|^2) \leq O(|V|^4)$

### Доказательство принадлежности NPC

Докажем в три шага:

$$\mathbf{NAE-3SAT} \stackrel{(1)}{\leq_p} \mathbf{Max\ Cut} \stackrel{(2)}{\leq_p} \mathbf{Min\ Cut\ into\ Bounded\ Sets} \stackrel{(3)}{\leq_p} \mathbf{Min\ k-Cut}$$

*Примечание: NP-полнота NAE-3SAT доказывалась на лекции*

*P.S. принадлежности к NP задач Max Cut и Min Cut into Bounded Sets будут опущены, т.к. являются относительно несложными. Доказательство также можно найти в [GJ]*

### Шаг 1

Строим  $f: x \in \mathbf{NAE-3SAT} \Leftrightarrow f(x) \in \mathbf{Max\ Cut}$

Пусть на вход поступила  $\phi$ . Пусть  $(x_1, x_2 \dots x_n)$  — переменные,  $(C_1, \dots C_m)$  — дизъюнкты. Тогда

- $V = \{x_1, \neg x_1, x_2, \neg x_2, \dots x_n, \neg x_n\}$
- $E = E_1 \cup E_2$ , где
  - $E_1 = \{(x_i, \neg x_i) \mid \forall i \in \{1, 2, \dots n\}\}$
  - $E_2 = \{\text{тройки ребер, соединяющие вершины из } C_i \mid \forall i \in 1, 2, \dots n\}$
- $C = 2m + n$

Доказательство корректности:

$\Rightarrow$ :

Положим  $\tilde{V}$  - истинные литералы из искомого набора (*см обозначения*),  $V \setminus \tilde{V}$  - ложные.

Т.к. в  $V$  треугольнике будут как истинные, так ложные литералы  $\Rightarrow$  ровно 2 его ребра будут находиться между  $\tilde{V}$  и  $V \setminus \tilde{V}$

Также каждая пара  $x, \neg x$  будет содержать ребро между  $\tilde{V}$  и  $V \setminus \tilde{V}$

Значит, всего ребер между  $\tilde{V}$  и  $V \setminus \tilde{V} \geq 2m + n$

$\Leftarrow$ :

Аналогично. Положим литералы  $\tilde{V}$  - истинные,  $V \setminus \tilde{V}$  - ложные. Докажем, что такое определение корректно и полученные набор - искомым для  $\phi$ .

Пойдем от противного. Пусть  $\neg x_i$  и  $x_i$  лежат в  $\tilde{V}$ . Тогда всего ребер между  $\tilde{V}$  и  $V \setminus \tilde{V} \leq n - 1 + 2m < C$ , т.к. между половинами может быть не более 2 ребер от каждый треугольника и одного ребра из оставшихся пар  $(x, \neg x)$  - противоречие. Это доказывает корректность определения набора

Аналогично доказывается, что построенный набор - искомый: если в какой-то тройке все литералы одинаковые  $\Rightarrow$  полностью лежит в  $\tilde{V}$  или  $V \setminus \tilde{V} \Rightarrow$  ребер между  $\tilde{V}$  и  $V \setminus \tilde{V} \leq 2(m - 1) + n < C$

Полиномиальность  $f$  можно считать очевидной

### Шаг 2

В этом абзаце будет шляпка (  $\hookrightarrow$  ), будет значить "объект из Min Cut into Bounded Sets", без шляпки - Max Cut Строим  $f$ :

На вход поступил  $G$  и  $C$ . Тогда

- Пусть  $|V| = n$
- $\hat{C} = K_{2n}$  - полная клика на  $2m$  вершинах
- $w(e) = \begin{cases} 1, \text{ если } e \in E \\ 2, \text{ если } e \notin E, \text{ т.е. ребро было добавлено} \end{cases}$
- $\hat{C} = 2n^2 - C$
- $\hat{B} = n$

Доказательство корректности:

$\Rightarrow$ :

Рассмотрим разрез  $(S, V \setminus S)$  из Max Cut. Распределим новые вершины по долям, дополнив их до размера  $n$  (к первой добавим  $n - |S|$ , ко второй  $n - |V \setminus S|$ ).

Нетрудно убедиться в том, что величина разреза  $\leq 2n^2 - C = \hat{C}$

$\Leftarrow$ :

Рассмотрим две доли размера  $n$  -  $V_1, V_2$ . Если между ними разрез стоимости  $\leq \hat{C} \Rightarrow$  разрез в исходном графе  $\geq 2n^2 - \hat{C} \Rightarrow (V_1 \cap V, V_2 \cap V)$  - искомое разбиение Max Cut

Полиномиальность  $f$  следует из  $|\hat{C}| \leq 4 \cdot |G|$

### Шаг 3

*Min Cut into Bounded Sets - частный случай Min k-Cut*

На вход поступили  $G, C, B, w$ . Тогда

- $f = id$
- $k = 2$

$\Rightarrow$ :

Разрез в Min Cut into Bounded Sets является искомым разрезом в Min 2-Cut.

$\Leftarrow$ :

Разрез в Min 2-Cut является искомым разрезом в для Min Cut into Bounded Sets (условие на  $|V_{1,2}| \leq B$  выполнено по определению  $f$ ).

## Итог

**Min k-Cut**  $\in$  *NPC*

## Алгоритм нахождения k-разреза, не хуже удвоенного оптимального:

### Алгоритм [SV] EFFICIENT

#### Описание алгоритма без оптимизаций

- Пронумеруем вершины в графе  $V = \{1, 2, \dots n\}$ .
  - Построение дерева Гомори-Ху:
    - Нахождение матрицы  $F: F_{ij}$  - максимальный поток (он же минимальный разрез) между вершинами  $i, j$
    - Нахождение в графе  $(V, F)$  макс остова - дерева  $(V, \tilde{E})$ , где  $\tilde{E} \subset F$  и  $\sum_{e \in \tilde{E}} w(e)$  - максимальна
    - Полученное дерево и будет деревом Гомори-Ху
  - Сортировка ребер в дереве ГХ в порядке возрастания
  - Набор ребер (жадным алгоритмом), пока не образуется k компонент связности (*сумма весов объединения набранных разрезов - финальный ответ*)

#### Оптимизации & итоговая сложность

- Шаг 1**
  - [GoHu] п.1 можно выполнить за  $n - 1$  запусков поиска *max flow* вместо  $n^2$  и без нахождения макс остова.
  - Поиск *max flow* может быть выполнен за разные ассимптотики. В силу простоты реализации в проекте выбран алгоритм Эдмондса-Карпа с сложностью  $O(|V| \cdot |E|^2)$
- Шаг 2**
  - Обычная сортировка за  $O(|V| \log |V|)$
- Шаг 3**
  - $O(|V|)$  в худшем случае

Таким образом, общая сложность задачи составит  $O(|V|^2 |E|^2)$  - полиномиальная сложность

*Замечание: нетрудно убедиться, что алгоритм без оптимизаций также имеет полиномиальную сложность. Однако, для тестирования будет реализована версия с оптимизациями. Ее корректность (шаг 1) можно найти в [GoHu]*

#### Корректность алгоритма

Все доказательства ниже можно найти в оригинале [SV]

#### Основное свойство дерева Гомори-Ху

$\forall i, j \ min\_cut(i, j)$  - минимальное ребро на пути  $i \leftrightarrow j$  в дереве Гомори-Ху

#### Утверждение 1

$\forall i, j, v \ F_{ij} \geq min(F_{iv}, F_{vj})$

▲ Рассмотрим минимальный разрез между  $i, j$  и ту долю, в которой оказалась  $v$ . Тогда выполнено хотя бы одно из слдующих неравенств:  $\begin{cases} F_{ij} \geq F_{iv} \\ F_{ij} \geq F_{vj} \end{cases}$

#### Утверждение 2

$\forall i, j, v_1, v_2 \dots v_m \ F_{ij} \geq min(F_{iv_1}, F_{iv_2}, \dots F_{iv_{m-1}}, F_{v_m}, F_{vj})$

▲

Доказательство аналогично утв 1: найдутся  $v_l$  и  $v_{l+1}$  лежащие в разных долях разреза

■

Доказательство основного свойства ГХ:

▲

Рассмотрим путь  $i \rightarrow v_1 \rightarrow v_2 \dots v_m \rightarrow j$  из  $i$  в  $j$ .  $(v_l, v_{l+1})$  его кратчайшее ребро. Тогда верны следующие неравенства:

- $F_{ij} \geq F_{iv_{l+1}}$  - утв 2.
- $F_{ij} \leq F_{iv_{l+1}}$  - свойство максимального остова  $F_{ij} > F_{iv_{l+1}}$  означало бы, что при замене ребра  $(v_l, v_{l+1})$  на  $(i, j)$  в дереве  $(V, \tilde{E})$  суммарный вес бы вырос, а связность осталась)

■

#### Обозначения:

$wt(C)$  - вес разреза (суммарный вес ребер, его пересекающих) в исходном графе  $G$

$b_i$  -  $i$ -ый разрез (набор ребер), выбранный в ходе работы жадного алгоритма (некий минимальный разрез между некоторой парой вершин), **ребра которого не содержатся в  $b_1 \cup b_2 \cup b_3 \dots \cup b_{i-1}$**

Разрезы, выбранные алгоритом -  $B = b_1 \cup b_2 \cup \dots \cup b_l$

Если при удалении  $B$  образовывается больше  $k$  компонент, удалим лишние ребра из  $b_k$ , уменьшив число компонент, тем самым также уменьшив  $wt(B)$

Пусть  $A$  - оптимальный разрез.  $V_1, V_2, \dots V_k$  - компоненты, полученные при удалении  $A$ .

$a_i$  - разрез (ребра) между  $V \setminus V_i$  и  $V_i$

Будем считать  $wt(a_1) \leq wt(a_2) \dots \leq wt(a_k)$

По алгоритму  $wt(b_1) \leq wt(b_2) \dots \leq wt(b_l)$

Следствие:

$$2wt(A) = wt(a_1) + wt(a_2) + \dots wt(a_k)$$

$$wt(B) \leq wt(b_1) + wt(b_2) + \dots wt(b_l) \quad (*)$$

В отличие от первого, во втором нельзя использовать равенство, т.к.  $b_i$  могут пересекаться между собой

#### Утверждение 3

$l \leq k - 1$

▲

Достаточно проверить, что любой разрез, (впервые) удаляющий какое-то ребро из исходного графа, увеличивает число компонент. Тогда каждый разрез  $b_i$  увеличил число компонент хотя бы на 1.

Действительно, пусть  $b_i$  не увеличил число компонент. Рассмотрим ребро  $(u, v)$ , удаленное разрезом  $b_i$ , но не разрезами  $b_1, b_2, \dots b_{i-1}$ . Т.к. ребро пересечено разрезом  $b_i \Rightarrow$  вершины  $u, v$  оказались в разных компонентах после удаления  $b_i$ . При этом до удаления были соединены ребром  $\Rightarrow$  находились в одной компоненте.

■

Теперь вместо  $l$  разрезов будем считать, что в ходе алгоритма было выбрано ровно  $k - 1$ : разрез  $b_l$  будет выбран с кратностью  $l$ , если его добавление создало  $l$  новых компонент. Перенумеруем их как  $b_1, b_2, \dots, b_{k-1}$ .  $(*)$  останется выполнено

#### Определение

Пусть  $c_1, c_2, \dots c_{2^l}$  - всевозможные разрезы упорядоченные по весу:

$$wt(c_1) \leq wt(c_2) \dots wt(c_{2^l})$$

Назовем упорядочивание **согласованным перечислением**, если разрезы  $b_i$  в нем сохраняют порядок. Другими словами

Если  $j_1: b_i = c_{j_1}$ , Тогда  $j_1 \leq j_2 \leq \dots \leq j_{k-1}$

*Комментарий: из-за того, что веса  $b_i$  могут совпадать, не любое перечисление будет согласованным*

#### Лемма

Рассмотрим любое согласованное перечисление всех разрезов графа  $G - \{c_i\}_{i=1}^{2^l}$

Тогда для любого индекса  $j$  выполнено:

$$c_1 \cup c_2 \cup \dots \cup c_j = b_1 \cup b_2 \cup \dots \cup b_q \quad (*)$$

где  $b_q$  — последний из разрезов, чей индекс в перечислении не превосходит  $j$ . Т.е.  $\begin{cases} j_q \leq j \\ j_{q+1} > j \end{cases}$

▲

От противного. Найдем такой **минимальный**  $j$ , для которого неверно  $(*)$ . Очевидно,  $j \geq 1$ , т.к. для  $\emptyset = \emptyset$  для  $j = 0$  (также положим  $b_0 = \emptyset$ ).

Среди всех согласованных перечислений выберем то, в котором  $j$  **максимально**. Дальнейшее доказательство будет вестись для данного перечисления.

Т.к.  $j$  - минимально  $\Rightarrow (*)$  было выполнено для  $j - 1$ :

$$c_1 \cup c_2 \cup \dots \cup c_{j-1} = b_1 \cup b_2 \cup \dots b_q$$

#### Первый случай:

$j = j_q$ , т.е.  $c_j$  совпало с  $b_q$ . Тогда для  $j - 1$  было верно

$$c_1 \cup c_2 \cup \dots \cup c_{j-1} = b_1 \cup b_2 \cup \dots \cup b_{q-1}$$

А следовательно верно и  $(*)$

#### Второй случай:

$j > q \Rightarrow$  для  $j - 1$  верно

$$c_1 \cup c_2 \cup \dots \cup c_{j-1} = b_1 \cup b_2 \cup \dots \cup b_q$$

Пусть  $t := b_{q+1}$ , т.е. на данный момент есть цепочка

$$b_q \leq j - 1 < j < t = b_{q+1}$$

Тогда сравним веса  $wt(c_j) \leq wt(c_t)$ . Заметим, что равенства  $wt(c_j) = wt(c_t)$  быть не может, т.к. иначе поменяв местами в перечислении  $t$  и  $j$  мы бы получили все также согласованное перечисление, но с большим  $j$ , для которого не выполнено  $(*)$  - противоречие.

Значит, верно  $\begin{cases} wt(c_j) < wt(c_t) = wt(b_{q+1}) \\ c_1 \cup c_2 \cup \dots \cup c_j \neq b_1 \cup b_2 \cup \dots \cup b_q \Rightarrow c_j \not\subset b_1 \cup b_2 \cup \dots \cup b_q \end{cases}$

Рассмотрим ребро  $e \in c_j \setminus b_1 \setminus b_2 \dots \setminus b_q$ . Обозначим минимальный разрез для вершин-концов ребра  $e$  за  $C$ . Тогда в дереве Гомори-Ху было ребро  $\hat{e}$  с весом  $wt(C) \leq wt(c_j) < wt(b_{q+1})$ , причем к моменту выбора  $j + 1$ -го ребра  $\hat{e}$  не было выбрано (по определению ребра  $\hat{e}$  без шляпки) - противоречие с алгоритмом (вместо разреза  $b_{j+1}$  должен был быть выбран разрез  $C$ ).

■

#### Определение

Для каждого  $i \leq index(a_{k-1})$  в списке  $\{c_j\}$  положим

- $p_i$  - число  $a_j$ , что  $a_j$  встречается в перечислении до  $c_i$
- $A_i$  - объединение разрезов  $a_j$ , что  $a_j$  встречается в перечислении до  $c_i$
- $q_i$  - число  $b_j$ , что  $b_j$  встречается в перечислении до  $c_i$
- $B_i$  - объединение разрезов  $b_j$ , что  $b_j$  встречается в перечислении до  $c_i$

Докажем 2 коротких утверждения:

#### Утверждение 4

Для любого  $i$  верно, что количество компонент при удалении  $B_i - comps(B_i) \geq comps(A_i)$  - количество компонент при удалении  $A_i$

▲

Из леммы выше следует  $A_i \subset B_i = \cup_{j=1}^i c_j$ , откуда следует  $comps(B_i) \geq comps(A_i)$

■

#### Утверждение 5

$comps(A_i) \geq p_i + 1$

$comps(B_i) \geq q_i + 1$

▲

Пусть  $a_1, a_2 \dots a_{p_i}$  те разрезы, что попали в  $A_i$ . Тогда после их удаления попарно несвязанными окажутся  $V_1, V_2, \dots V_{p_i}$  и остальные вершины, откуда следует  $comps(A_i) \geq p_i + 1$

Здесь важно ограничение на  $i \leq index(a_{k-1})$ , чтобы оставшиеся вершины были не пустым множеством

Пожим образом доказываемы и  $comps(B_i) = q_i + 1$

Если некоторый разрез  $b_q$  добавляет  $t$  компонент связности при его удалении, то и в  $q_i$  он будет входить с кратностью  $t$ .

■

#### Следствие из утверждений 4, 5

$q_i \geq p_i$

▲

Из утверждений 4,5 верна цепочка

$$q_i + 1 = comps(B_i) \geq comps(A_i) \geq p_i + 1$$

■

#### Заключение доказательства

По следствию выше на начальном отрезке  $\{c_1, c_2 \dots c_k\}$   $b$ -разрезов встречается не меньше, чем  $a$ -разрезов.

Тогда  $index(b_k) \leq index(a_k)$  и отрезке  $\{c_1, c_2 \dots c_k\}$   $b$ -разрезов встречается для  $i = index(b_k) - 1$

Но тогда  $\forall i < k \overset{\text{упорядоченность } wt(c)}{\Rightarrow} wt(b_i) \leq wt(a_i)$